

퀄리저널 개발 파일 설명서

1. 주요 디렉터리와 코드 구조

프로젝트 전체 구조: QualiJournal 관리자(Admin) 시스템의 디렉토리 구조는 다음과 같으며, 각 구성 요소의 역할은 아래와 같습니다 ¹ ² :

- `server_quali.py` - FastAPI 기반 백엔드 서버의 진입점으로, 관리자용 API들을 제공하는 메인 어플리케이션입니다 ³. Cloud Run 컨테이너가 실행되면 Uvicorn/Gunicorn을 통해 이 모듈의 앱(app)을 구동합니다 (기본 8080포트) ⁴.
- `index.html` - 관리자 대시보드 프론트엔드로 사용되는 단일 HTML 페이지입니다 ⁵. Vanilla JS로 구현되었으며, 여러 **버튼**과 **슬라이더 UI** 등을 통해 백엔드 API를 호출해 기능을 수행합니다 (예: “일일 보고서 생성”, “키워드 요약/번역”, “선정보 요약/번역”, “Export MD/CSV” 등) ⁶. 이 파일은 Cloud Run 배포 시 static 파일로 제공되며, **관리자 인증 토큰**이 유효할 경우에만 정상적인 API 호출이 가능하도록 설계되었습니다.
- `config.json` - 시스템 동작 설정값을 저장하는 구성 파일입니다. 예를 들어 **게이트 임계값**(최소 필요한 승인 기사 수), 기능 토글(특정 기능 활성화 여부) 등이 정의됩니다 ⁷. 서버 기동 시 이 파일을 불러와 설정을 로드하며, 운영 중 임계값 조정 등의 변경사항은 이 파일에 영구 저장됩니다 ⁸.
- `tools/ 디렉터리` - 뉴스 수집 및 발행 관련 **유틸리티 스크립트** 모음입니다 ⁹. 자동화 파이프라인의 각 단계나 편집 보조 작업을 개별 파이썬 스크립트로 구현하였으며, 필요에 따라 수동 실행하거나 서버에서 호출합니다:
 - `make_daily_report.py` - 현재까지 **승인된 기사 목록**(`selected_articles.json`)을 모아 일일 뉴스 보고서(마크다운)를 생성하는 스크립트 ¹⁰.
 - `enrich_cards.py` - 수집된 기사들(`selected_keyword_articles.json`)에 대해 **영문 요약문을 생성**하고, 번역 API 키가 설정된 경우 한국어 번역도 수행하여 JSON 필드에 반영하는 스크립트 ¹¹.
 - `rebuild_kw_from_today.py` - 당일 수집된 원본 여러 JSON들을 표준 구조로 병합하여 **작업본**(`selected_keyword_articles.json`)을 생성합니다 ¹².
 - `augment_from_official_pool.py` - 승인된 기사 수가 부족할 경우 **공식 뉴스풀**에서 추가 기사를 채워 전체 기사 수를 20개 수준으로 보정하고 승인 기사도 최소 15개 확보합니다 ¹².
 - `force_approve_top20.py` - 기사 점수 등 정렬 기준으로 **상위 20개 기사에 일괄 승인 처리**를 합니다 ¹².
- (이외에도 보조 스크립트들): `sync_selected_for_publish.py` (작업본에서 승인된 기사만 추출해 발행본 갱신), `repair_selection_files.py` (JSON 구조가 어긋났을 때 형식을 교정) 등이 있습니다 ¹³.
- `data/ 디렉터리` - (선택 사항) JSON 데이터 파일을 저장하는 경로입니다 ¹⁴. 운영 시 **기사 수집 결과와 편집본 JSON**을 이 폴더에 둘 수 있습니다. 주요 파일:
 - `selected_keyword_articles.json` - “작업본”: 하루 키워드에 대해 수집된 **모든 기사 목록**과 그 메타데이터를 포함하는 JSON 파일입니다 (기사 제목, 요약, 출처, 날짜, 점수, `approved` 여부, `editor_note` 등) ¹⁵. 편집자는 이 파일을 기반으로 기사를 선별하고 `approved=true`로 표시하며 필요 코멘트를 추가합니다 ¹⁶.
 - `selected_articles.json` - “발행본”: 작업본에서 `approved=true`인 기사만 추린 최종 발행 대상 JSON입니다 ¹⁷. 최종 뉴스 페이지 생성 시 이 파일의 기사들만 사용됩니다. (발행본은 작업본과 동일한 구조

로 날짜와 기사 배열을 포함합니다.)

(주의: 현재 시스템에서는 DB 대신 이들 JSON 파일로 데이터를 관리하며, 편집 완료 후 발행 시점에 두 파일의 내용이 일치하도록 유지해야 합니다.)*

- `archive/` **디렉토리** - 발행된 결과물들을 날짜별로 저장하는 **아카이브 경로**입니다 ¹⁸. 예를 들어 `archive/reports/`에는 매일 생성된 **뉴스 보고서**의 Markdown 및 HTML 파일이 저장되고, `archive/enriched/`에는 요약/번역된 기사 카드의 Markdown이 저장됩니다 ¹⁸ ¹⁹. 또한 JSON 원본과 발행물의 과거 기록들을 보존하여 필요시 재검토하거나 복구할 수 있습니다 ²⁰ ²¹. 발행 시 **동일한 내용의** `.json`, `.md`, `.html` **파일을** 모두 남겨 두며, 파일명은 `YYYY-MM-DD-<키워드>-report.md` 혹은 `..._selected.md` 등의 형식을 따릅니다 ²² ¹⁹.
- `tests/` **디렉토리** - Pytest 기반의 **통합 테스트 스크립트**가 위치합니다 ²³. 주요 API와 기능에 대한 단위/통합 테스트를 포함하며, CI 파이프라인에서 배포 전에 테스트를 수행하여 코드 안정성을 검증할 수 있습니다. (예: 토큰 인증이 설정된 경우 테스트 클라이언트에 토큰을 주입하는 등 세팅 필요 ²⁴.)
- `Dockerfile` - Google Cloud Run 배포를 위한 **컨테이너 이미지 설정 파일**입니다 ²⁵. Python 베이스 이미지에 필요한 패키지를 설치하고, `uvicorn/gunicorn`을 통해 `server_quali:app`을 `0.0.0.0:8080`에서 실행하도록 CMD를 지정합니다 ⁴. Cloud Run 환경에 맞춰 `$PORT` 환경변수를 사용하고, 컨테이너 시작 시 최신 코드를 실행할 수 있게 설정합니다.
- `.env` **파일** - 개발 및 배포 시 사용되는 **환경변수 설정 파일**로, `ADMIN_TOKEN`, `API_KEY`, `DB_URL`, `GCS_BUCKET` 등 민감정보와 설정값을 키=값 형태로 포함합니다 ²⁶. 이 파일은 Git에 커밋되지 않으며(**비공개**), 로컬 개발 및 CI에서는 이 파일을 참조하고 Cloud Run 배포시에는 동일 값이 환경변수로 주입됩니다 ²⁷ ²⁸.
- `.github/workflows/ci.yml` - GitHub Actions 기반 **CI/CD 파이프라인 정의 파일**입니다 ²⁹. 코드가 `main` 브랜치에 푸시될 때 자동으로 도커 이미지 빌드 -> 테스트 -> Cloud Run 배포까지 수행되도록 설정되어 있습니다 ³⁰ ³¹. (아래 CI/CD 섹션 참조)

2. 파일 간 의존성과 정합성 분석

QualiJournal 시스템의 여러 파일과 모듈은 **뉴스 수집부터 발행까지의 흐름** 속에서 긴밀히 연계되어 있습니다. 올바른 동작을 위해서는 각 파일의 내용과 상태가 서로 일관성을 가져야 하며, 중복 정의나 충돌이 없도록 관리해야 합니다. 주요 의존성과 정합성 이슈는 다음과 같습니다:

- **작업본 vs. 발행본 JSON의 연계**: 수집 단계에서 생성된 `selected_keyword_articles.json` (작업본)과 최종 발행에 사용하는 `selected_articles.json` (발행본)은 내용적으로 연속된 관계를 가집니다 ¹⁷. 즉, 작업본에서 편집자가 승인(`approved=true`)한 기사만을 발행본에 반영해야 하며, 두 파일의 **포맷과 구조**는 동일하게 유지됩니다. 이를 위해 **동기화 스크립트**(`sync_selected_for_publish.py`)를 제공하여 작업본에서 승인된 기사들만 추출해 발행본을 갱신합니다 ³². 또한 발행본의 구조가 혹시 어긋날 경우 `repair_selection_files.py`를 통해 표준 딕셔너리 형태로 교정함으로써, **데이터 정합성**을 유지합니다 ¹³.
- **품질 게이트(게이트 임계값) 일관성**: 시스템은 하루 발행을 위해 최소 15개의 기사가 승인되어야 하는 **품질 게이트** 조건을 갖고 있으며, 이 기준은 설정 파일과 코드 전반에서 통일되어야 합니다. 설정값 `features.gate_required` (예: 15)가 `config.json`에 저장되고, 서버 메모리에도 로드되어 `/api/status` 응답 등에 활용됩니다 ³³. 운영 중 관리자가 **KPI 슬라이더**를 통해 이 값을 조정하면 (`PATCH /api/config/gate_required`), 서버는 메모리의 값과 `config.json`의 값을 동시에 업데이트하고, 관련 모듈(예: 외부 RSS 수집기의 `gate_required` 변수 등)도 함께 갱신하여 **설정 간 불일치가 없도록** 처리합니다 ⁸. 이렇게 변경된 임계값은 즉시 `/api/status` API의 출력(`gate_required`, `gate_pass`)에 반영되어 관리자 UI에 나타나게 됩니다 ³⁴.

• **데이터 중복 및 충돌 방지:** 현재 구현에서는 단일 JSON 소스가 진실의 원천(Single Source of Truth) 역할을 합니다. 모든 수집 결과는 작업본 JSON 하나로 통합되고, 발행본도 하나의 JSON으로 정제됩니다¹⁷. DB를 도입하지 않은 대신 JSON 파일에 의존하므로, 중복된 데이터 소스나 스키마 충돌은 적은 편입니다. 다만, 동시에 여러 프로세스가 JSON을 갱신할 경우 충돌 가능성이 있으므로, 운영 시 한 번에 하나의 경로(예: UI나 스크립트 중 하나)를 통해서만 편집/발행을 진행해야 합니다. 예를 들어 GUI 환경에서 승인 처리 중에 별도의 스크립트를 병행 실행하지 않도록 하는 운영 수칙이 필요합니다. 또한, JSON 편집 시 문법 오류나 필드 누락이 발생하면 이후 파이프라인 단계에서 오류를 일으킬 수 있으므로, `repair_selection_files.py` 등을 활용해 포맷을 검증/교정해야 합니다¹³.

• **다중 인스턴스 환경 고려:** 현 시스템은 기본적으로 단일 Cloud Run 인스턴스에서 동작하도록 가정되어 있습니다. 만약 부하 증가 등으로 Cloud Run이 여러 인스턴스를 동시에 실행하면, 각 인스턴스는 자신만의 로컬 파일 사본을 가지므로 설정이나 데이터가 서로 일치하지 않을 수 있습니다³⁵. 예를 들어 한 인스턴스에서 `config.json`의 값을 변경해도 다른 인스턴스에는 반영되지 않을 수 있고, JSON 데이터 역시 인스턴스별로 따로 관리됩니다. 이러한 문제를 해결하려면 공유 DB나 중앙 Config 서비스 도입이 필요하며, 현재는 단일 인스턴스 운용을 전제로 일관성을 유지하고 있습니다³⁵. 운영자가 이를 인지하고, 동시 인스턴스가 생기지 않도록 Cloud Run 설정(동시 처리 요청 수 제한 등)이나 트래픽 조절을 하는 것이 좋습니다.

• **환경변수와 설정의 정합성:** `.env` 파일과 Cloud Run에 설정된 환경변수들은 코드 내 참조되는 키들과 맞아야 합니다. 예를 들어 `ADMIN_TOKEN`, `API_KEY`, `DB_URL`, `GCS_BUCKET` 등의 키는 동일한 이름으로 설정되어야 하며, 하나라도 누락되면 관련 기능이 동작하지 않습니다^{36 37}. 특히 `ADMIN_TOKEN`은 서버 기동 시 필수로 로드되므로 배포 환경에 빠뜨리지 않도록 해야 하고, `API_KEY` 역시 번역 기능 사용 시 필요합니다³⁸. 이런 환경 설정값은 GitHub Secrets와 Cloud Run 변수에 동일하게 반영되어야 하며, 변경 시 두 곳을 모두 업데이트하여 불일치를 방지해야 합니다^{39 40}.

• **기존 워크플로우와의 연계:** QualiJournal 시스템 개발 초기에 존재했던 수동 파이프라인(오케스트레이션)과 현재 Admin 시스템 간의 역할 분담도 고려해야 합니다. 과거에는 PowerShell 스크립트(`run_quali_today.ps1`)나 명령행 도구(`orchestrator.py`)를 통해 수집-재구성-보충-발행 일련의 과정을 순차 실행했지만^{41 42}, 최종 구축된 Admin 서버는 이러한 흐름을 API 엔드포인트 호출로 대체했습니다. 결과적으로 중복되는 로직은 대부분 제거되었으며, `tools/` 내 스크립트를 Admin API가 내부적으로 호출하는 형태로 통합되었습니다²². 다만, 혹시라도 개발 중 과거 스크립트를 수동으로 실행하고 동시에 Admin UI에서 기능을 수행하면 동일 작업을 이중 수행하거나 데이터 경합이 일어날 수 있으므로, 한 가지 방법만 선택적으로 사용해야 합니다. 현재는 Admin UI를 통한 작업을 권장하며, 옛 수동 스크립트 기반 실행 방식은 예외적 상황(예: 긴급 백업 발행)에서만 활용하도록 합니다.

3. 핵심 기능(API) 흐름과 연결 관계

QualiJournal 시스템의 핵심 기능은 뉴스 데이터의 수집 → 요약 및 번역 → 편집 승인 → 최종 보고서 생성 및 배포까지의 흐름으로 이루어져 있습니다. 관리자 UI와 백엔드 API, 그리고 백엔드의 내부 스크립트들이 유기적으로 연결되어 이러한 파이프라인을 구현합니다. 각 주요 기능의 흐름을 단계별로 살펴보면 다음과 같습니다:

• ① **데이터 수집 (Collector):** 편집자가 하루 키워드를 선정하면 해당 키워드와 관련된 다양한 출처(공식뉴스 RSS, 블로그, 학술 DB, 커뮤니티 등)에서 관련 콘텐츠를 자동 수집합니다^{43 44}. 이 단계는 QualiJournal 코드베이스 내에서 FastAPI API로 구현되지 않고, 별도의 크롤러/오케스트레이션 스크립트로 구현되어 있습니다. 예를 들어 `orchestrator.py`를 `--collect` 옵션으로 실행하거나, 각 출처별 수집 함수를 호출하여 모든 기사를 모읍니다⁴⁴. 수집된 원본 JSON들은 임시적으로 로컬에 저장된 후, `rebuild_kw_from_today.py` 스크립트를 통해 표준화된 하나의 JSON(작업본)으로 병합됩니다⁴⁵. 현재 Admin UI에는 수집 시작을 트리거하는 별도 버튼이나 API가 없으며, 이 단계는 백엔드 외부에서 수행되거나 사전에 스케줄러에 의해 돌도록 구성됩니다 (예: 매일 특정 시각 Cloud Scheduler가 수집 잡(Job)을 실행) - 운영자는 수집 완료 후부터 Admin UI를 활용합니다.

• ② **요약 및 번역 (Summarize/Translate):** 수집이 완료되고 편집자가 초기 기사 목록을 확보하면, Admin UI를 통해 **요약/번역 기능**을 수행할 수 있습니다. 관리자 대시보드의 “키워드 요약/번역” 버튼을 누르면 `POST /api/enrich/keyword` 엔드포인트가 호출되며, 서버는 내부적으로 `tools/enrich_cards.py` 스크립트를 실행합니다¹¹. 이 스크립트는 작업본 (`selected_keyword_articles.json`)의 **모든 기사들에 대해 핵심 요약문(영어)**을 생성하고, 환경변수 `API_KEY`로 번역 API 키가 제공된 경우 **한국어 번역문도** 생성합니다^{11 46}. 생성된 요약/번역 결과는 해당 JSON에 필드로 추가되거나 갱신되며, 처리 완료 후 Admin UI에는 성공 메시지가 표시됩니다. 만약 일부 기사에서 요약 실패나 번역 API 오류(예: 쿼리 초과)가 발생해도, 스크립트는 가능한 한 나머지 기사들에 대해 결과를 채우고 **Markdown 요약본**을 생성하도록 구현되어 있습니다^{47 11}. 오류 발생 시 응답 JSON에 error 메시지가 포함되며, UI 로그 영역에 출력되어 운영자가 원인을 파악할 수 있게 합니다⁴⁸ (예: 번역 키 미설정 시 "Translation API key not set" 에러 표시).

• ③ **기사 선별 및 승인 (Editorial Approval):** 요약까지 완료된 후, 편집자는 **어떤 기사를 최종 발행할지** 선정합니다. 현재 QualiJournal Admin에는 기사 목록을 GUI로 편집하는 기능이 완전히 구현되어 있지 않아, 편집자는 JSON 파일을 직접 열어 `approved` 플래그를 true로 설정하거나, 필요하면 `force_approve_top20.py` 같은 스크립트를 사용해 자동 선정을 할 수 있습니다. **발행 기준(게이트)**은 기본적으로 최소 15건 승인이며, 이 임계값 미달 시 발행이 불가능하도록 UI에서 “일일 보고서 생성” 버튼이 비활성화되거나, 또는 발행 시도 시 에러를 반환합니다⁴⁹. 다만 긴급한 경우를 대비해 시스템은 해당 조건을 무시하고 발행할 수 있는 메커니즘도 갖추고 있습니다. 예를 들어 발행 단계에서 승인된 기사가 15개 미만이면 `require_editor_approval` 설정을 일시적으로 false로 전환해 강제로 보고서를 생성하고, 바로 다시 true로 복구하는 처리가 가능하도록 구현되었습니다⁵⁰. (이러한 조치는 자동 파이프라인에서의 예외 상황을 다루기 위한 것이며, 일반 운영에서는 편집자가 15개 이상을 승인하도록 유도합니다.)

• ④ **선정본 요약 및 최종 검수:** 편집을 마쳐 승인된 기사 목록(`selected_articles.json`)이 확정되면, 발행 직전 단계로 “선정본 요약/번역” 기능을 수행할 수 있습니다. 이는 Admin UI의 해당 버튼 클릭으로 `POST /api/enrich/selected` 엔드포인트를 호출하여 실행되며, 다시 한번 `enrich_cards.py`를 사용하지만 이번에는 **발행본 JSON의 기사들만 대상으로 요약/번역 필드를 보강**합니다⁵¹. 이 작업은 최종 발행될 기사들의 내용이 충분히 요약/번역되어 있는지 확인하고, 추가적인 Markdown 요약 파일도 생성합니다¹⁹. 결과물로 `archive/enriched/YYYY-MM-DD_<키워드>_SELECTED.md` 파일이 만들어지며, 최종 선정된 기사들의 요약/번역본을 Markdown 형태로 포함합니다¹⁹. 이 파일은 내부 검수용이지만, 필요 시 발행물의 부록으로 활용하거나 향후 데이터베이스 적재 시 사용될 수 있습니다.

• ⑤ **일일 보고서 생성 및 발행 (Report Generation & Publish):** 모든 준비가 완료되면, 편집자는 Admin UI의 “일일 보고서 생성” 버튼을 눌러 **최종 뉴스레터 페이지**를 생성합니다. 이때 `POST /api/report` 엔드포인트가 호출되며, 서버는 내부적으로 `tools/make_daily_report.py` 스크립트를 실행합니다²². 이 스크립트는 발행본 JSON(`selected_articles.json`)에 담긴 모든 승인 기사들을 순서대로 모아 하나의 **Markdown 형식 보고서**를 생성합니다²². 생성되는 보고서에는 보도자료 형태의 제목, 각 기사 카드(제목, 요약, 출처, 날짜, 편집자 코멘트 등), 그리고 전체를 맥락 있게 소개하는 머리말 등이 포함됩니다 (세부 구성은 기획에 따라 작성). 스크립트 실행 성공 시 `archive/reports/YYYY-MM-DD_<키워드>_report.md` 파일이 생성되고, 동일 내용의 HTML 파일도 저장됩니다²². 응답으로는 보고서 생성 성공 여부와 생성된 파일 경로 또는 다운로드 링크를 반환하며, Admin UI는 사용자에게 “보고서 생성 완료” 메시지와 함께 **다운로드 링크**를 표시합니다^{52 53}. 만약 스크립트 실행 중 예외가 발생하더라도 백엔드는 이미 수집된 JSON 데이터를 직접 읽어 최소한의 Markdown 뼈대(report)를 만들어내도록 **이중 안전장치**를 갖추고 있어, 어떠한 경우에도 보고서 파일은 얻을 수 있게 설계되어 있습니다⁵⁴. (예: `make_daily_report.py` 내 변수명 오류로 예외가 발생해도 백엔드가 자체 로직으로 JSON을 처리해 MD 생성)⁵⁵.

• ⑥ **결과물 확인 및 Export:** 보고서 생성 후 Admin UI에서는 “Export MD” 및 “Export CSV” 기능을 통해 산출물을 내려받을 수 있습니다. `GET /api/export/md` 엔드포인트 호출 시 최신 Markdown 보고서 파

일을, `GET /api/export/csv` 호출 시 선정된 기사 목록을 CSV로 변환하여 각각 제공하며, 브라우저에서 곧바로 다운로드가 시작됩니다 ⁵⁶ ⁵⁷. CSV에는 주요 필드(제목, 출처, 요약 등)가 컬럼으로 포함되어 있어, 추후 데이터 분석이나 DB 마이그레이션 시 유용하게 활용할 수 있습니다 ⁵⁸ ⁵⁹. Export 요청 역시 인증 토큰을 필요로 하며, 이를 UI가 자동으로 포함하여 호출합니다 ⁶⁰.

- ⑦ **배포 및 사용자 공개:** 최종적으로 생성된 Markdown/HTML 보고서는 **일반 사용자들에게 공개되어야** 합니다. 현재 QualiJournal 시스템은 **Cloud Run 백엔드가 관리자용**으로 보호되어 있으므로, 생성된 결과물을 일반 사용자용 웹에 게시하는 작업은 별도의 단계로 이루어집니다. 보고서 생성이 완료되면, 시스템은 **백업/배포용 GCS(Google Cloud Storage)** 버킷에 해당 `.md`, `.html`, `.json` 파일을 업로드합니다 ⁶¹ ⁶². 이 버킷은 아카이브 겸 배포 저장소로 사용되며, 올바른 IAM 권한이 설정된 서비스 계정으로 Cloud Run 컨테이너가 GCS에 쓰기 가능하도록 구성되어 있습니다 ⁶³. 업로드된 파일은 운영팀이 지정한 **최종 사용자 웹 도메인**(예: `standardai.co.kr`)을 통해 서비스되는데, 구체적인 방법으로는 정적 호스팅 또는 콘텐츠 복사 방식이 활용됩니다 (아래 9번 섹션 참조). 예를 들어 GCS 버킷에 업로드된 HTML 페이지를 Static Website로 설정하고 사용자 도메인에 매핑하면, 일반 독자들은 `standardai.co.kr`에서 해당 콘텐츠를 볼 수 있습니다.

요약하면, **관리자(Admin) 시스템**은 수집된 데이터를 요약/번역하고 편집자 승인 과정을 거쳐 하루 한 키워드에 대한 **큐레이션 뉴스 페이지**를 생성하는 역할을 합니다 ⁶⁴. 이 전체 과정이 일원화된 UI와 API 흐름으로 연결되어 있으며, 중간 산출물들은 JSON/Markdown/HTML 형태로 남겨져 **백업 및 서비스 활용**에 제공됩니다.

4. Cloud Run 배포 구조, ENTRYPOINT 및 스케줄러/시크릿 구성

QualiJournal Admin 서버는 **Docker 컨테이너**로 패키징되어 Google Cloud Run에 배포됩니다 ³⁰. Cloud Run 환경에서의 배포 구조 및 관련 설정은 다음과 같습니다:

- **Docker 이미지 및 ENTRYPOINT:** 배포 시에는 프로젝트 루트의 `Dockerfile`을 통해 이미지를 빌드합니다. 이 Dockerfile은 Python 기반 이미지를 사용하고, 소스 코드를 복사한 뒤 `pip install`로 필요한 패키지를 설치하며, 최종적으로 컨테이너 실행 명령(CMD)으로 **Uvicorn 서버 구동**을 설정합니다. 예를 들어 `CMD ["uvicorn", "server_quali:app", "--host", "0.0.0.0", "--port", "8080"]` 형태로 ENTRYPOINT를 정의하여 FastAPI 앱을 실행합니다 ⁴. Cloud Run은 컨테이너 포트로 기본 8080을 사용하므로, 코드와 Uvicorn 설정 모두 이 포트에 맞춰져 있습니다. (참고로 성능 및 멀티쓰레드를 위해 Gunicorn+Uvicorn Workers 조합을 쓸 수도 있지만, 현재 트래픽 규모에서 Uvicorn 단일 프로세스로도 충분합니다.)
- **Cloud Run 서비스 구성:** QualiJournal Admin 컨테이너는 GCP 프로젝트 내 특정 **서비스 이름**으로 Cloud Run에 배포됩니다 (예: `qualijournal-service` 혹은 `admin`) ⁶⁵. 배포 지역은 지연 시간을 고려해 서울 리전(`asia-northeast3`)으로 하고, **동시 접속자 수가 많지 않으므로** CPU 1개, 메모리 512MB~1GB 정도로 설정되었습니다 ⁶⁶. Cloud Run 배포 시 `--allow-unauthenticated` 옵션을 사용하는데, 이는 외부에서 토큰 없는 요청도 Cloud Run 서비스에 접속 가능하게 함을 의미합니다 ⁶⁷. 하지만 **보안은 애플리케이션 레벨**(ADMIN_TOKEN 검증)에서 이뤄지므로, Cloud Run 레벨에서는 인증을 열어두어도 문제없다는 판단입니다 ⁶⁷.
- **환경변수 및 시크릿 관리:** Cloud Run 서비스에는 앞서 `.env`에 정의된 주요 환경변수들이 그대로 등록되어야 합니다. 배포 방법에 따라 두 가지 방식이 활용되었습니다:
- **소스 배포의 경우:** `gcloud run deploy` 명령 시 `--update-env-vars` `"ADMIN_TOKEN=...,API_KEY=...,DB_URL=...,GCS_BUCKET=..."` 옵션을 사용하여 값을 주입합니다 ⁶⁵. 이때 각 키=값 쌍을 쉼표로 구분하며, 배포 후 해당 값들이 Cloud Run **Variables & Secrets** 탭에 정상 반영되었는지 확인합니다 ⁶⁸ ⁶⁹.

- CI/CD 배포의 경우: GitHub Actions 워크플로우에서 `env_vars` 인자를 통해 Secrets에 저장된 값을 전달합니다 ^{70 71}. 예를 들어 `{{ secrets.ADMIN_TOKEN }}` 형태로 ADMIN_TOKEN 값을 주입하고, GCP 서비스 계정 JSON 키도 `{{ secrets.GCP_SA_KEY }}` 로 받아와 사용합니다 ^{70 39}. Actions에서 제공하는 공식 Cloud Run 배포 액션(`google-github-actions/deploy-cloudrun`)을 활용하여 이미지와 환경변수 세트를 함께 배포하며, commit SHA 등을 메타데이터 라벨로 설정해 버전 추적을 용이하게 합니다 ^{72 73}.
- Secret Manager 활용: 민감한 값은 GCP Secret Manager에 저장 후 Cloud Run에 시크릿 참조로 주입할 수도 있습니다. 예를 들어 배포 명령에 `--set-secrets="API_KEY=projects/<PRJ>/secrets/<NAME>:latest"` 옵션을 주면 해당 시크릿의 최신 버전이 환경변수로 설정됩니다 ⁷⁴. 현재는 간소화를 위해 직접 `env_vars`로 넣지만, **장기적으로 Secret Manager를 사용하는 것이 권장됩니다.**
- **CI 파이프라인과 연계:** Cloud Run 배포는 **GitHub Actions**를 통해 자동화되어 있습니다. main 브랜치에 코드가 머지되면 CI 파이프라인이 트리거되고, Docker 이미지 빌드 → 이미지 푸시(Artifact Registry) → Cloud Run 배포를 순차적으로 수행합니다 ^{30 31}. 이를 위해 GCP IAM에서 Cloud Run 배포 권한이 있는 서비스 계정(예: `{{ secrets.GCP_SA_KEY }}`)을 만들고, 그 키를 GitHub Secrets에 저장해 워크플로우에서 활용합니다 ⁷⁰. Workflow YAML에는 프로젝트 ID(`GCP_PROJECT`), 서비스명(`CLOUD_RUN_SERVICE`), 리전(`GCP_REGION`) 등이 하드코딩 또는 Secrets/Env로 지정되어 있으며, ADMIN_TOKEN 등도 Secrets로 불러와 env로 전달합니다 ^{39 71}. CI Job 중에는 (선택적으로) Pytest로 **테스트를 실행**하도록 해, 테스트 실패 시 배포를 중단시키는 안전장치도 넣을 수 있습니다 ⁷⁵. 최종 배포 완료 후 Actions 로그에 Cloud Run 서비스 URL을 출력하고, 필요시 Slack 알림 등 추가 액션을 설정할 수 있습니다 ⁷⁶.
- **스케줄러(일정 실행) 구성:** QualiJournal의 “하루 한 키워드” 발행은 **일일 주기 작업**입니다. 이를 자동화하기 위해 GCP **Cloud Scheduler**를 이용한 배치 실행을 고려할 수 있습니다. 예를 들어 Cloud Scheduler가 매일 아침 특정 시각에 Cloud Run 서비스의 `/api/report` 엔드포인트를 호출하도록 설정하면 (인증 토큰 헤더 포함), 정해진 시간에 자동 발행이 가능합니다. 또는 별도의 **Cloud Run Job**(배치 잡)을 구성해 orchestrator 스크립트를 주기 실행할 수도 있습니다. 현재 구현 단계에서는 완전한 무인 자동발행까지는 구축되지 않았지만, 운영 가이드에는 **자동화 고려 사항**이 언급되어 있습니다. 예컨대 “백업 파일 존재 확인: GCS 아카이브 버킷에 어제 날짜의 파일이 저장되었는지 체크. 없으면 자동 발행 실패로 간주하고 수동 발행 필요”와 같은 내용으로 보아 ⁷⁷, 추후 Cloud Scheduler 기반의 자동 발행을 계획하고 점검 체계를 마련한 것으로 보입니다. 요약하면, **스케줄러는 현재 완전 구현되지 않았으나**, Cloud Run 환경에서 트리거식 배포나 CI 일정 실행으로 발전시킬 수 있는 부분입니다.
- **Cloud Run 도메인 및 트래픽:** 배포된 Cloud Run 서비스는 기본적으로 Google이 제공하는 도메인(예: `https://qualijournal-service-xxxxxx.run.app`)으로 접근할 수 있고, 이는 **HTTPS가 기본 적용**됩니다 ⁷⁸. 운영 편의를 위해 **커스텀 도메인**(`admin.standardai.co.kr`)을 Cloud Run 서비스에 매핑하였으며, TLS 인증서도 적용해 안전하게 통신합니다 ⁷⁸. 배포 시 최신 리비전에 트래픽 100%를 할당하여, 새로운 버전이 올라가면 즉시 모든 요청이 그쪽으로 가도록 설정합니다. 만약 Cloud Run이 새 리비전 생성 후 트래픽을 자동 전환하지 않는 설정일 경우(수동 승인이 필요한 설정), 운영자는 `gcloud run services update-traffic --to-latest` 명령이나, Actions 내 추가 단계를 통해 **최신 리비전에 트래픽을 모두 할당**하도록 합니다 ^{79 80}.
- **배포 후 검증:** 운영자는 새 코드가 배포될 때마다 Cloud Run **서비스 상세 정보**를 조회해 이미지 태그나 digest에 최신 커밋 해시가 반영되었는지 확인합니다 ^{81 82}. 또한 Cloud Run 리비전의 `metadata.labels`에 자동으로 붙는 `commit-sha` 라벨이 GitHub의 최신 커밋과 일치하는지 검증하여, 실제 배포된 컨테이너가 해당 커밋 기반임을 추적합니다 ^{73 82}. 배포 후 **로그 모니터링**도 필수입니다 - Cloud Run 로그를 실시간으로 보면서 새 컨테이너 시작 시 에러 traceback, 의존성 로드 문제, DB 연결 오류 등이 없는지 확인해야 합니다 ⁸³. 만약 심각한 오류가 발생하면 해당 리비전을 **롤백**하거나 빠르게 패치 후 재배포합니다.

다 ⁸³ . 이와 함께, Cloud Run에 주입된 환경변수(특히 ADMIN_TOKEN, API_KEY 등)가 제대로 설정되었는지도 콘솔의 Variables 탭이나 `gcloud run services describe` 출력으로 재차 검증합니다 ^{28 84} .

요약하면, Cloud Run 배포 구조는 **컨테이너화된 FastAPI 앱 + 환경변수 구성 + CI/CD 자동화 + 커스텀 도메인**의 형태로 구성되어 있으며, 개발자는 코드 푸시만으로 배포까지 이뤄지는 **GitOps 워크플로우**를 구현해 두었습니다. 또한 GCP의 여러 기능(Secret Manager, Cloud Scheduler 등)을 활용할 수 있는 여지를 남겨 두어, 향후 **보안성과 자동화 수준을 높일 수 있도록** 설계되었습니다.

5. 사용자 UI와 관리자 UI의 기능 흐름 및 연계

QualiJournal 시스템은 **관리자(Admin) UI**와 **일반 사용자 UI**가 분리되어 있으며, 각기 다른 목적과 기능 흐름을 갖습니다. 여기서는 두 UI의 역할과 동작, 그리고 양자 간 연계되는 지점들을 설명합니다.

• 관리자 UI (Admin Dashboard):

관리자 UI는 `admin.standardai.co.kr` 도메인에서 제공되는 **편집자용 대시보드**입니다. 단일 HTML 페이지(`index.html`)로 구성되며, **Vanilla JavaScript**로 필요한 상호작용을 구현했습니다 ⁸⁵ . 페이지가 로드되면 우선 편집자 인증을 위해 **토큰 입력**을 요구할 수 있습니다 ⁸⁶ . (최초 접근 시 자바스크립트를 통해 ADMIN_TOKEN을 물어보거나, 브라우저 localStorage 등에 저장된 이전 토큰을 불러와 사용할 수 있습니다.) 올바른 토큰을 입력하면 이후 모든 API 요청에 해당 토큰이 Authorization 헤더로 자동 포함되며, 편집자는 대시보드 기능을 이용할 수 있게 됩니다 ⁸⁷ .

UI 상에는 **주요 지표(KPI) 표시 영역**과 **액션 버튼들**, 그리고 로그/메시지 출력 영역이 존재합니다. KPI 영역에는 현재 시스템의 상태 지표 - 예: 수집된 총 기사 수, 승인된 기사 수, 기사 상태별 카운트(candidate/ready/rejected) 및 커뮤니티/공식뉴스 수, 현재 게이트 임계값과 통과여부 등 - 가 표시됩니다 ³³ . 이 값들은 주기적으로 백엔드 `/api/status`를 호출하여 실시간 갱신되며, 새로운 기사가 승인되거나 임계값이 변경될 때 즉각 업데이트 됩니다 ^{88 8} . 액션 버튼들은 앞서 설명한 핵심 기능들을 트리거하며, 각 버튼 클릭 -> 해당 API 호출 -> 응답 처리 -> 결과 표시의 흐름으로 동작합니다 ⁸⁵ . 예를 들어 “일일 보고서 생성” 버튼을 누르면 `/api/report` POST 요청이 보내지고, 성공 시 UI에 다운로드 링크가 나타나며 실패 시 에러 메시지가 화면에 붉은색으로 출력됩니다 ^{89 90} . 이 때 UI는 **로딩중 표시**(스피너 또는 “...생성 중” 문구)를 띄워 사용자가 진행 상황을 인지하게 하고, 중복 클릭을 방지하기 위해 버튼을 잠시 비활성화합니다 ^{91 92} . 모든 버튼들은 이러한 UX 고려사항(로딩 피드백, 중복클릭 방지, 오류 안내)을 반영하여 구현되었으며, 이는 최종 패치 과정에서 개선되었습니다 (아래 패치 적용 여부 참조).

관리자 UI와 백엔드의 연계는 **REST API 호출**로 이루어집니다. Admin UI가 실행되는 도메인(origin)에 대해서만 백엔드가 CORS 요청을 허용하도록 화이트리스트 설정되어 있어, **교차 도메인 요청 보안도** 확보합니다 ⁹³ . (예: admin UI가 구동되는 `admin.standardai.co.kr` 만 허용, 그 외 origin에서 온 요청은 차단) 이는 설정된 토큰이 유출되더라도 승인된 도메인에서만 API 통신이 가능하게 해주는 **이중 보호** 장치입니다 ⁹³ . 요약하면, **관리자 UI는 편집 workflow에 맞춘 다양한 기능 제공과 보안 강화**에 초점을 맞추고 있습니다. 관리자는 이 UI를 통해 데이터 요약/번역부터 발행까지 대부분 작업을 수행하고, 시스템이 생성한 결과물을 다운로드받아 확인하거나 별도 사용자 사이트에 게시하는 작업을 이어가게 됩니다.

• 일반 사용자 UI (Reader UI):

일반 사용자 UI는 최종 뉴스 콘텐츠를 소비하는 **독자 대상 웹 인터페이스**입니다 ⁶⁴ . QualiJournal의 컨셉이 “하루 한 키워드 역사 뉴스”인 만큼, 사용자 UI에서는 특정 키워드에 대한 기사들의 모음(특별호)을 **타임라인 형태로** 보여주는 것이 핵심입니다 ^{64 94} . 예를 들어 편집자가 “인공지능”이라는 키워드를 선택해 발행했다면, 사용자 UI에서는 그 키워드와 관련된 역사적 기사들이 과거부터 최신까지 **연대기 순으로 배열**되어, 독자로 하여금 해당 주제의 흐름을 한눈에 보도록 합니다 ⁶⁴ . 기사 카드는 편집자가 승인한 핵심 기사 15건 이상으로 구성되고, 각각 제목, 요약문(국/영), 출처, 날짜, 그리고 (있다면) 편집자 코멘트가 포함됩니다 ⁹⁵ . 디자인 원칙은 **쉽고 직관적**인 UX로, 복잡한 메뉴 없이도 독자가 스크롤하며 뉴스를 읽을 수 있도록 단순한 레이아웃을 취합니다 ^{96 97} .

사용자 UI의 구현 측면: 현재 사용자 UI는 Admin과 완전히 분리된 별도 프론트엔드로, static 페이지 또는 별도 서비스로 제공될 수 있습니다. Admin 시스템이 생성한 최종 결과물(HTML 또는 JSON)을 소스로 하여, 사용자 웹에 이를 게시하는 방식입니다. 가장 단순한 형태로는, Admin이 생성한 HTML 페이지를 정적 호스팅하여 `standardai.co.kr`에서 제공하는 것입니다. 실제 운영에서 `standardai.co.kr` 도메인은 해당 HTML을 가리키도록 설정되어, 별도 웹서버 없이 **GCS 버킷의 공개 파일**을 서빙하거나 Cloud CDN을 통해 캐시하게 할 수 있습니다. 이 경우 사용자 UI는 특별한 서버 로직이 없고, 미리 생성된 정적 페이지를 보여주지만 합니다. 사용자는 접속 즉시 완성된 뉴스 페이지를 보게 되고, 추가 상호작용은 필요하지 않습니다.

보다 발전된 방식으로는 **동적 사용자 UI**를 개발하는 것입니다. 예를 들어 React/Vue 기반의 SPA로 만들어, 여러 날짜의 키워드 뉴스를 조회하거나 검색, 소셜 공유 등의 기능을 넣을 수 있습니다. 이때 백엔드로부터 데이터를 가져와야 하므로, Admin 시스템의 JSON 아카이브를 읽는 **공개 API** 또는 별도 DB가 필요합니다. 현재 Admin API는 인증이 걸려 있어 일반 사용자에게 직접 데이터를 주지는 않으므로, 만약 동적 UI를 도입한다면 **별도의 Public API** 엔드포인트를 두거나, GCS에 업로드된 JSON 파일들을 AJAX로 불러오는 방식을 고려할 수 있습니다. 예를 들어 `standardai.co.kr/latest.json` (GCS에 공개)에서 최신 발행본 데이터를 가져와 렌더링하는 것입니다. 다만 이러한 구현은 아직 계획 단계이며, 현 시점의 사용자 UI는 **정적 페이지 공개** 수준으로 운영되고 있는 것으로 보입니다. (사용자 UI 개선 가이드 문서에서도 원칙과 방향성이 제시되어 있을 뿐, 구체 코드 구현은 향후 과제로 남겨둔 상태 64.)

- **Admin UI와 User UI의 연계 포인트:** 두 UI는 직접적인 연결고리는 없지만, **콘텐츠를 매개로** 연계됩니다. 관리자가 Admin UI에서 발행 작업을 완료하면 **생성된 뉴스 콘텐츠(HTML/JSON)**가 사용자 UI 쪽으로 전달되어야 하는데, 이 부분은 운영 프로세스로 해결됩니다. 현재 주된 방법은 **파일 기반 연계**입니다: Admin 시스템이 결과물을 GCS에 올리고, 사용자 사이트(`standardai.co.kr`)가 그 파일을 가리키도록 하는 것이죠 61 20. 예를 들어 GCS 버킷에 `2025-10-26_키워드.html`이 업로드되면, `standardai.co.kr`에서 해당 날짜에 대한 링크를 제공하거나 바로 그 파일을 메인 페이지로 serve하도록 설정합니다. 이러한 연계는 CI/CD처럼 자동화될 수도, 혹은 일부분 수동으로 이뤄질 수도 있습니다. 현재 문서상으로 **완전 자동 게시까지 구현되었다는 언급은 없어**, 운영자가 최종 HTML을 확인한 뒤 메인 사이트에 반영(예: 파일을 특정 경로로 복사하거나, WordPress 등에 임베드)하는 단계를 거칠 수 있습니다 21 98. 다만, GCS에 이미 공개되어 있다면 링크만 걸어주면 되므로, **반영은 거의 실시간으로** 가능할 것입니다.

요약하면, **관리자 UI**는 내부 편집 workflows를 지원하고, **사용자 UI**는 외부 독자들에게 결과를 보여주는 역할로 분리되어 있습니다. 두 UI 사이의 연결은 결과 콘텐츠를 통해 간접적으로 이루어지며, 토큰 등 **보안 경계가 분명하게** 구분되어 있습니다 99. 향후에는 사용자 UI에서 **과거 아카이브 검색**이나 **키워드 제한** 등의 상호작용 기능이 추가될 수 있고, 이 때는 Admin 측에서 제공하는 공개 API나 DB 지원이 필요하게 될 것입니다.

6. 오류 및 미완성/비연결 요소 분석

프로젝트 최종본을 검토한 결과, 대부분 핵심 기능은 구현되고 통합되었으나 몇 가지 **미완성되었거나 연결되지 않은 요소**, 그리고 **잠재적 오류 가능성**이 발견되었습니다:

- **기사 승인 UI의 부재:** Admin UI에는 KPI 통계와 작업 트리거 버튼들은 존재하지만, 정작 **개별 기사들을 열람하고 선택(approve)하는 UI 컴포넌트는 존재하지 않습니다**. 즉, 편집자가 어떤 기사를 발행할지 선별하는 작업은 현재 **JSON 파일 편집** 또는 `force_approve_top20.py` 등의 스크립트에 의존합니다. 이 부분은 명백히 **미완성 기능**으로 보입니다. 원래 기획 의도는 Admin UI에서 수집된 기사 리스트를 표 형태로 보여주고, 체크박스로 승인하거나 코멘트를 입력하는 등 인터페이스를 제공하는 것일 텐데, 개발 시점에 우선순위에 밀려거나 시간 부족으로 구현되지 못한 것으로 판단됩니다. 따라서 현 운영에서는 편집자가 `selected_keyword_articles.json`을 수동으로 열어 `approved:true`로 표시하는 방식을 사용해야 하며, 이는 기술적으로 가능하지만 UX상 불편합니다. 이 요소는 **차후 개선 필요** 사항입니다 (예: JS로 기사 리스트 뿌려주기 및 클릭 승인 기능 추가).

- **데이터 수집 트리거의 부재:** 앞서 언급했듯 Admin UI에는 “데이터 수집 시작”에 해당하는 버튼이나 API가 없습니다. 하루 한 키워드에 대한 크롤링/수집은 여전히 Admin 시스템 외부에서 이루어져야 합니다. 예컨대, 운영자가 하루 키워드를 결정한 후 별도 터미널에서 수집 스크립트를 돌리거나, Cloud Scheduler가 orchestrator를 실행해 주어야 합니다. Admin 시스템과 이 수집 프로세스가 **연결되지 않아서** 발생하는 간극이 존재하며, 이 역시 미완성 연계 부분입니다. 결과적으로 수집 단계에서 오류가 나거나 데이터가 제대로 안 들어와도 Admin UI에는 표시가 되지 않고, 편집자가 직접 JSON을 확인해야 합니다. (이론적으로 `/api/status`에 `selection_total`이 0으로 나오는 것으로 파악 가능하나, 명시적이지 않음) 추후 개선으로 Admin UI에 “수집 시작” 버튼 및 진행 상황 표시, 혹은 **수집 자동화 모니터링** 기능이 추가되어야 전체 워크플로우가 UI상에서 닫힐 것입니다.
- **다중 사용자/권한 기능 미구현:** 현 Admin 시스템은 **단일 관리자 토큰**으로 전체를 보호하는 방식이며, 편집자별 계정이나 권한 레벨 구분 기능은 없습니다 ⁹⁹. 만약 향후 운영 인력이 늘어나서 여러 명이 동시에 시스템을 써야 하거나 권한(예: 관리자 vs 검수자)을 나누어야 할 경우를 대비한 구조가 마련되어 있지 않습니다. 데이터베이스에 사용자를 관리하는 `users` 테이블 등을 만들 수 있겠지만 (문서상 아이디어로만 언급 ¹⁰⁰), 아직 구현이 전혀 되어 있지 않습니다. 따라서 현 시점에서는 **모든 운영자가 동일한 토큰을 공유**하여 접속해야 하므로 보안상 다소 위험하고 추적도 어렵습니다. 이 부분은 **미구현 기능**으로, JWT 인증이나 OAuth 연동, 혹은 Organization SSO 등의 도입을 통해 개선 가능하다고 문서에서 제안하고 있습니다 ⁹⁹.
- **데이터베이스 미도입으로 인한 제약:** 시스템은 **파일 기반(JSON)**으로 작동하기 때문에, **실시간 업데이트나 여러 키워드 히스토리 누적 관리**에 한계가 있습니다. 예를 들어 과거 발행된 키워드들의 기록을 누적해 검색하거나 비교하려면, JSON 파일들을 일괄 로딩하여 처리해야 하므로 비효율적입니다. 문서에서는 추후 관계형 DB나 NoSQL 도입을 통해 데이터 모델을 정규화/최적화할 것을 권장하지만 ¹⁰⁰ ¹⁰¹, 현재 코드에서는 DB 연결 (`DB_URL`)이 설정돼 있어도 활용되지 않습니다. 이는 미연결된 구성 요소로, DB를 사용하도록 코드를 확장하려면 데이터 적재, 조회 API 등 추가 개발이 필요합니다. DB를 쓰지 않는 현 구조에서는 **동시성 이슈**도 존재합니다 (앞서 언급한 다중 인스턴스 문제 등). 요약하면 DB 관련 설정과 코드 구조는 일부 마련됐지만 실제 **DB 연동 로직은 구현 안 됨**이므로, 이 역시 미완성 부분입니다.
- **토큰 인증 우회 옵션 제안 (적용 안 됨):** 개발 도중 논의된 내용 중 하나로, “일반 사용자 UI와 구분되어 현재 Admin 시스템은 단일 토큰으로 보호되지만, 내부 사용자만 쓰는 경우 일부 엔드포인트에 토큰 검증을 생략할 수 있다”는 아이디어가 있었습니다 ⁶⁰ ⁹⁹. 특히 Export 같이 편의 기능은 토큰 없이도 쓸 수 있게 하자는 제안이었으나, **보안 원칙상 적용되지 않았습니다**. 즉, 최종 코드에서 **모든 API 엔드포인트는 여전히 ADMIN_TOKEN 검증을 거치며**, 토큰 없이 호출하면 401/403을 반환합니다 ¹⁰². 따라서 “토큰 해제”에 대한 패치는 이행되지 않았고, Admin Token 기반 인증 체계는 그대로 유지되고 있습니다. 이는 완성된 기능이지만, 질문에서 언급된 “토큰 해제”는 아마 해당 제안의 적용 여부를 뜻하는 것으로 보이며, 결론적으로 **미적용**임을 밝힙니다.
- **UI 세부 기능 보완 필요점:** 프론트엔드 코드 상에서 일부 **이벤트 미바인딩**이나 UX 세부사항이 누락된 부분들이 있었습니다만, 최종 패치를 통해 대부분 개선되었습니다. 예를 들어 과거에는 버튼 클릭 시 중복 요청 방지 처리가 없어 연타하면 이중 수행될 위험이 있었고, 진행 중임을 나타내는 로딩 인디케이터도 없었습니다. **최종 버전(v2.3)**에서는 이러한 부분을 보완하여, 각 버튼 클릭 시 JS가 버튼을 비활성화하고 로그 창에 “...중” 메시지를 띄우는 등 **이벤트 바인딩 및 UI 피드백**이 구현되었습니다 ⁹¹ ¹⁰³. 다만, 혹시 남아있는 사소한 이슈로 **UI 요소의 레이아웃이나 브라우저 호환성** 문제가 있을 수 있습니다. (예: 모바일 대응 미흡, 긴 제목의 처리 등은 디자인 가이드에만 있고 구현은 추후 과제). 이러한 부분은 운영 중 피드백을 받아 계속 다듬어야 할 것입니다.
- **외부 API 오류 처리:** 요약/번역에 쓰이는 외부 API (예: 번역 API)의 경우, **쿼터 초과나 키 유효성 오류** 발생 시 처리를 어떻게 할지가 중요합니다. 현재 구현에서는 해당 오류를 잡아내 UI에 메시지를 띄워주긴 하지만 ⁴⁸, 근본적인 대처(예: 쿼터 관리나 재시도 로직)는 마련되어 있지 않습니다. 따라서 번역 API 키의 **요금제나 제한을 모니터링**하지 않으면 어느 날 번역 기능이 동작하지 않을 위험이 있습니다. 이 부분은 운영상 주의점이자, 필요하면 캐시나 다중 API 키 지원 등의 개선 여지가 있습니다.

정리하면, **캘리저널 시스템은 전반적으로 완성도 높게 구현되었으나**, 기사 승인 UI, 데이터 수집의 UI 통합, 다중 사용자 지원, DB 기반 구조 등 몇몇 핵심 기능이 아직 연결되지 않았거나 향후 개발이 필요합니다. 또한 UI/UX 세부 개선과 외부 API 안정성 대책 등은 **운영 단계에서 꾸준히 보완**해나갈 영역입니다.

7. 패치 적용 여부 (예: KPI 슬라이더, 토큰 해제 등)

개발 산출물에서 확인된 최근 패치 사항들을 정리하면 다음과 같습니다:

- **KPI 슬라이더 적용:** 네, **게이트 임계값 조정 슬라이더** 기능이 최종 Admin UI에 구현되어 있습니다. 이는 프론트엔드에서 range input으로 제공되며, 사용자가 슬라이더를 움직일 때 백엔드의 `PATCH /api/config/gate_required` 엔드포인트를 호출하게 됩니다 ⁸. 해당 API 구현은 전달받은 value로 서버 메모리의 `gate_required` 값을 업데이트하고, 동시에 `config.json` 파일의 설정도 변경하여 영구 저장합니다 ⁸. 변경 직후 `/api/status`의 응답에 새로운 `gate_required`와 `gate_pass`가 반영되어 UI KPI에 표시되고, 편집자는 승인 필요 기사 수를 즉각 조절할 수 있습니다 ³⁴. 이 기능은 개발 막바지에 패치로 추가된 것으로 보이며, 실운영시 편집자의 요구(“승인 15건 제한을 유연하게 바꾸고 싶다”)를 반영한 것입니다. 따라서 KPI 슬라이더 패치는 **정상 적용**되어 있습니다.

- **Admin 토큰 관련 패치:** 질의에 언급된 “토큰 해제”는 앞서 6번 항목에서 설명한 대로, Admin API의 인증을 완화하는 방안입니다. 결론적으로 해당 패치는 **적용되지 않았으며**, 현재도 모든 Admin API는 **Authorization 헤더의 토큰 검증을 통과**해야만 동작합니다 ¹⁰⁴. 개발 도중 Export 편의나 내부망 한정 사용 시 토큰을 생략하는 옵션이 거론됐으나, 원칙적으로 모든 API에 인증을 거는 것이 안전하다는 판단 하에 유지된 것입니다 ⁶⁰. 다만, 코드에는 토큰을 URL 파라미터로 전달받는 방안 등도 코멘트로 논의되었으나 보안 우려로 채택되지 않았습니다 ¹⁰⁵ ¹⁰⁶. 결론적으로 “토큰 해제”는 **미적용**이며, Admin Token 체계는 유지되고 있습니다 (운영중 토큰을 바꾸거나 없앨 경우 반드시 서버 재배포 필요).

- **UI 이벤트 처리 패치:** 최종 버전 Admin UI (`index.html`)에는 여러 **UX 개선 패치**가 이루어졌습니다. 로그를 통해 확인된 v2.3 변경사항에는 “admin index.html에서 UI 이슈 수정”이 포함되어 있었으며 ¹⁰⁷, 실제 기능으로 확인된 것들은 다음과 같습니다:

- 버튼 중복 클릭 방지: 한 작업이 진행 중일 때 동일 버튼을 다시 눌러도 요청을 또 보내지 않도록 JS에서 클릭 이벤트 발생 시 해당 버튼을 `disabled=true` 처리하고, 응답이 오면 다시 활성화합니다 ⁹¹ ¹⁰⁸.
- 로딩 상태 표시: API 호출 후 응답이 올 때까지 사용자에게 피드백을 주기 위해, 예를 들어 “보고서 생성 중...”과 같은 문구를 화면의 로그 영역에 표시하거나, 버튼에 로딩 스피너 CSS를 적용하는 등의 패치가 이뤄졌습니다 ⁹² ¹⁰³. 이를 통해 수 초 이상 걸리는 작업에서도 사용자 경험을 향상시켰습니다.
- 에러 메시지 출력 개선: 과거에는 오류 발생 시 개발자 콘솔에만 로그가 찍혀 운영자가 원인을 알기 힘들었으나, 이제 UI 화면에 붉은색 경고 메시지로 친숙한 문구를 표시합니다 ⁸⁹ ⁹⁰. 예를 들어 인증 실패 시 “인증 오류: 관리 토큰을 확인하세요”, 번역 API 실패 시 “보고서 생성 실패: 번역 API 오류” 등 구체적인 안내가 나타나도록 패치되었습니다.

- 기타: Export 기능 개선(토큰을 URL 대신 헤더로 보내도록 변경), CSV 한글 깨짐 방지 안내, 슬라이더 조작 시 실시간 값 표시 등 자잘한 UI 향상도 이루어졌습니다 ¹⁰⁹ ⁹¹.

- **기능 패치 여부 확인 - KPI 슬라이더 & Token:** 위 내용 요약하면, **KPI 슬라이더 패치**는 코드에 반영되어 **UI와 API가 연동 완료**되었고, **토큰 해제 패치**는 적용되지 않아 현행 토큰 인증 유지, **UI 이벤트 관련 패치**들은 대부분 반영 완료되었습니다. 따라서 질문에서 예시로 든 두 항목 중 슬라이더는 “적용 O”, 토큰 해제는 “적용 X”로 구분할 수 있습니다. 이외에 개발 막바지에 발견된 몇몇 버그 (예: index.html에서 잘못된 변수 참조 등)는 CI 테스트를 통해 인지되어 수정되었으며 ¹¹⁰, 최종 결과물에는 치명적 버그가 없는 것으로 판단됩니다.

8. CI/CD 및 GitHub Actions 구성 여부

QualiJournal 프로젝트는 **CI/CD 파이프라인**이 구축되어 있으며, GitHub Actions를 이용해 코드 변경부터 배포까지 자동화하고 있습니다 ². 구성 내용은 다음과 같습니다:

- **GitHub Actions 워크플로우 존재:** `.github/workflows/ci.yml` 파일이 레포지토리에 포함되어 있고, 이는 **Main 브랜치에 대한 CI/CD 정의**입니다 ²⁹. 주요 트리거는 `push` 이벤트 (특히 main 브랜치)이며, 필요에 따라 PR 머지 시에도 동작하도록 설정되어 있습니다. 워크플로우 내부에는 **Jobs 단계**로 빌드, 테스트, 배포가 순서대로 명시되어 있습니다.

- **Docker 이미지 빌드 및 Artifact Registry 연동:** Actions가 시작되면 먼저 리포지토리 코드를 체크아웃하고 (`actions/checkout`), GCP 인증을 거쳐 (서비스 계정 키 JSON 이용) **Docker 빌드**를 수행합니다 ⁷⁰. 이때 GCP Artifact Registry 또는 Docker Hub에 **컨테이너 레지스트리** 자격증명을 설정하고, `docker build -t asia-northeast3-docker.pkg.dev/<PROJECT_ID>/containers/qualijournal:latest` 명령을 실행합니다 ¹¹¹. 빌드가 완료되면 `docker push`를 통해 이미지를 레지스트리에 업로드합니다 ¹¹². 이러한 과정은 CI 환경에서 비동기로 처리되며, Secrets에 저장된 GCP 프로젝트 정보와 Auth JSON이 활용됩니다.

- **자동 테스트 수행:** 워크플로우 중 Docker 이미지를 빌드한 후, **배포 전에 Pytest 등을 실행**하도록 구성되어 있습니다 ⁷⁵. 예를 들어 `pytest` 명령을 실행하여 `tests/` 디렉토리의 테스트가 모두 통과하는지 확인합니다. 만약 테스트 중 실패가 발생하면 Actions 워크플로우는 실패로 중단되어, **배포 단계로 넘어가지 않습니다** ⁷⁵. 이를 통해 **문제 있는 코드가 운영에 배포되는 것을 방지**합니다. (테스트에서는 토큰이 설정되어야 하므로, Actions job 환경에서 `ADMIN_TOKEN=test-token` 등 가짜 값을 넣어 테스트하기도 합니다 ²⁴.)

- **Cloud Run 자동 배포:** 테스트까지 성공하면, **Cloud Run 배포 단계**가 진행됩니다. YAML에 정의된 대로 `google-github-actions/deploy-cloudrun@v0` 액션이 사용되어, 앞서 빌드/푸시한 이미지 URL, Cloud Run 서비스 이름, 프로젝트 ID, 리전, 환경변수 등의 인자를 주어 **새 Revision을 배포**합니다 ⁷². 예컨대 `image: asia-northeast3-docker.pkg.dev/.../qualijournal:latest`, `service: admin` 등의 값을 넣고, `env_vars: ADMIN_TOKEN=${{ secrets.ADMIN_TOKEN }}`, ... 식으로 Admin 토큰 등도 함께 설정합니다 ^{71 39}. `deploy-cloudrun` 액션은 GitHub 제공 공식 액션으로, 배포 성공 시 해당 revision 정보를 출력할 수 있습니다. 또한 YAML에서 `with: traffic: 100` 등으로 지정하면 새 리비전에 100% 트래픽이 자동 할당됩니다 (혹은 기본값으로 latestRevision이 활성화됩니다).

- **Secrets 및 보안:** CI/CD 파이프라인에서 필요한 민감 정보들은 모두 **GitHub Secrets**에 저장되어 참조됩니다 ^{70 113}. 예를 들어 `GCP_SA_KEY` (JSON), `API_KEY`, `ADMIN_TOKEN`, `DB_URL`, `GCS_BUCKET` 등이 Secrets로 관리되고, workflow에서 `${{ secrets.VAR_NAME }}` 형태로 안전하게 사용됩니다 ³⁹. 덕분에 Repository 코드에는 민감 값이 하드코딩되지 않으며, 실수로 로그에 노출되지 않도록 세심히 처리했습니다 ^{114 38}. 운영 중에도 Admin Token 등의 교체가 필요할 경우 Secrets 값을 바꾸고 재배포하면 되므로 관리가 수월합니다 ¹¹⁵.

- **배포 후 검증 및 모니터:** Actions 워크플로우가 끝난 뒤, 운영자는 Cloud Run 콘솔이나 gcloud CLI로 **배포 결과를 검증**합니다. CI 설정상 Cloud Run에 `commit-sha` 라벨을 추가하도록 되어 있어, Cloud Run 서비스 describe 출력에 `commit-sha: <해시>`가 표시됩니다 ^{116 82}. 이 값이 GitHub의 최신 커밋과 일치하는지 확인하여, 배포된 컨테이너가 올바른 코드 기반인지 쉽게 파악 가능합니다 ¹¹⁷. 또한 Actions 로그에서도 마지막에 Cloud Run URL이 출력되므로, 스크립트 상 예시대로 `Invoke-WebRequest`로 `/index.html` 내용 일부를 가져와 **UI 버전 문자열(v2.3 등)이나 변경사항 반영 여부**를 확인할 수 있습니다 ^{118 119}. (실제로 Admin UI HTML 헤더에 버전 표시를 해두어 배포 확인에 쓰고 있습니다 ¹²⁰.) 이상이 없으면 배포 성공으로 간주하고 운영을 지속합니다.

- **요약:** CI/CD 구성은 문서화된 대로 정상 동작 중이며, GitHub Actions를 활용해 코드 -> 이미지 -> 배포의 자동화를 이뤘습니다 ^{29 30}. 개발자는 main 브랜치에 머지하기만 하면 몇 분 내 Cloud Run에 새로운 버전이 올라가며, 수동 개입 없이 서비스가 업데이트됩니다 ^{121 31}. 이로써 배포 일관성과 속도가 향상되었고, 잘못된 배포를 예방하기 위한 테스트/모니터링 체계도 갖추고 있습니다. CI/CD 파이프라인 자체도 보안에 유의하여 구성되었으므로 (Secrets 관리 등), 구성 여부 뿐 아니라 구성이 잘 되어 있음을 확인할 수 있었습니다.

9. 최종 도메인 기준 작동 흐름 (admin.standardai.co.kr vs standardai.co.kr)

QualiJournal 시스템은 최종 운영 단계에서 두 개의 주요 도메인을 통해 제공됩니다: - 관리자용 도메인: `admin.standardai.co.kr` - 사용자용 도메인: `standardai.co.kr`

각 도메인에서의 작동 흐름과 상호관계는 다음과 같이 정리됩니다:

- `admin.standardai.co.kr` (**관리자 사이트**): 이 도메인은 Cloud Run에 배포된 QualiJournal **Admin 서비스**와 연결되어 있습니다. GCP Cloud Run 커스텀 도메인 설정을 통해 `admin.standardai.co.kr` 요청이 앞서 배포한 컨테이너(`qualijournal-service`)로 라우팅됩니다 ⁷⁸. 접속 시 기본 경로(`/`)가 `index.html` 정적 파일을 서빙하여 관리자 UI가 로드됩니다 ⁵. 이후의 모든 상호작용(예: `/api/...` 요청)은 동일 도메인에서 AJAX로 호출되며, CORS 허용도 이 도메인에 대해서만 열려 있습니다 ⁹³. **보안 측면**에서, 운영자는 이 관리자 사이트 URL을 비공개로 관리하고, ADMIN_TOKEN을 알고 있는 내부자만 접근하게 합니다. 최초 접근 시 토큰 입력 페이지 또는 팝업이 나올 수 있으며, 올바른 토큰을 넣으면 본격적인 대시보드 기능을 사용할 수 있습니다 ⁸⁶. 만약 잘못된 토큰이면 서버는 401 Unauthorized를 반환하고, UI는 “인증 오류” 메시지를 보여줍니다 ^{104 90}.
관리자는 이 사이트에서 **일일 발행 작업을** 수행합니다. 통상 오전에 오늘의 키워드에 대한 수집이 끝나면 `admin.standardai.co.kr`에 접속하여 요약→검토→발행 일련 작업을 하고, 최종 Markdown/HTML을 다운로드 또는 확인합니다. **Cloud Run 상의 admin 서비스**는 배포 시 `allow-unauthenticated`로 설정되어 있기에 별도의 VPN 없이 인터넷 상에서 접근 가능하지만, 앞서 말한 토큰과 CORS 제한으로 보호됩니다 ^{67 93}.
운영 시나리오: 예를 들어 편집자 A가 아침 9시에 admin 사이트에 로그인하여, 전날 지정한 키워드 “탄소중립”의 수집 결과를 요약/번역하고, 15개 기사를 승인 표시한 뒤 보고서를 생성했다고 합니다. 보고서 생성 후 UI에 나타난 Markdown 다운로드 링크를 클릭하면 브라우저를 통해 `.md` 파일을 내려받을 수 있고 ^{56 57}, 해당 파일을 열어 제대로 작성됐는지 확인합니다. 문제가 없으면 이 파일(혹은 변환된 HTML)을 사용자 사이트에 올리기 위한 준비를 합니다.
- `standardai.co.kr` (**사용자 사이트**): 이 도메인은 일반 독자들이 접근하는 **컬리저널 뉴스 포털**입니다. 사용자가 이 주소로 접속하면, 당일 발행된 키워드 뉴스를 볼 수 있도록 페이지가 제공됩니다 ⁶⁴. 현재 구현에서 `standardai.co.kr`은 별도의 백엔드가 없는 **정적 호스팅** 형태일 가능성이 높습니다. Cloud Run의 admin 서비스와는 분리되어 있으며, 관리자 토큰 없이 공개되어야 하므로 같은 컨테이너를 쓸 수 없기 때문입니다. 가장 단순한 구성은, **GCS 버킷**(예: `qualijournal-archive`)에 저장된 HTML 페이지를 `standardai.co.kr` 커스텀 도메인에 **연결**시키는 것입니다 ^{122 62}. 실제로 GCP에서는 도메인을 Cloud Storage Static Website로 매핑하는 기능이 있고, 문서에서도 “Custom domain mapping (예: Cloud Run 커스텀 도메인 또는 정적 호스팅)을 적용할 수 있다”고 언급하고 있습니다 ¹²³. 만약 그렇게 설정되었다면, Admin이 GCS에 업로드한 `YYYY-MM-DD_<키워드>.html`이 곧 `standardai.co.kr`의 인덱스 페이지로 작동할 것입니다. (혹은 `index.html`로 복사되거나, 리디렉션 설정)

사용자 사이트의 동작: 사용자는 특별한 인증이나 절차 없이 `standardai.co.kr`에 접속하면 즉시 해당 날의 키워드 뉴스 페이지를 보게 됩니다. 그 페이지에는 15+개의 뉴스 카드가 시간 순으로 나열되고, 각 카드 클릭 시 해당 기사 원문 링크로 연결되거나, 카드 내에 요약본이 있으므로 클릭 없이도 내용을 파악할 수 있습니다. 디자인 상 모바일/데스크탑에서 모두 읽기 쉽도록 구성되어 있습니다 ^{96 97}. 사용자는 과거 날짜의 뉴스도 볼 수 있어야 할 텐데, 이것은 사이트 내에

서 링크로 제공하거나, 키워드별 아카이브 페이지를 두는 등 구현이 필요합니다. 현재 정적 방식이라면 매일 생성되는 HTML을 날짜별 URL로 쌓아두고, 메인에서는 최근 호만 보여줄 수도 있습니다. 예를 들어 standardai.co.kr 메인에 “오늘의 키워드: 탄소중립 (2025-10-26)”이 표시되고, 이전 뉴스 보기를 원하면 `/2025-10-25.html` 같은 링크를 누르는 식입니다. (향후 SPA로 개편된다면 API를 통해 목록을 불러오는 방식으로 개선될 수 있습니다.)

Admin → User 연결: 일반적으로 admin 사이트에서 생성한 결과물을 user 사이트에 반영하는 작업은 **자동 스크립트**로 처리하거나, 최소한 수동으로 복사/배치해야 합니다. Document 상에서는 Cloud Run Admin이 GCS에 업로드만 하면 “필요시 해당 MD 파일을 수동으로 내려받아 웹에 게시하거나”라는 언급이 있어 ²¹, 일부 수동 작업 여지를 남겨 두고 있습니다. 하지만 저희가 추론하기에, 이미 GCS 업로드와 도메인 연결을 해두었다면 수동 작업은 크지 않을 것입니다. 가령 admin 시스템이 GCS에 업로드를 끝내면 운영자가 standardai 웹을 리로드해 잘 보이는지만 확인하면 되는 정도일 수 있습니다. 만약 그렇지 않고 완전 분리라면, 운영자는 MD/HTML 파일을 내려받아 WordPress나 CMS에 올려야 할 수도 있지만, **QualiJournal**이라는 이름과 PDF 가이드 흐름상 그런 수동성을 도입했을 가능성은 낮아 보입니다. 어쨌든 현재 standardai.co.kr은 **Admin 시스템에서 나온 콘텐츠를 최종 노출**하는 역할이며, Admin과 직접 통신하거나 하는 일은 없습니다. Admin API는 토큰이 없어 호출 못하므로, user 사이트가 Admin 백엔드에 접근하는 일도 구조적으로 차단되어 있습니다 (REST API는 internal 용도). 이런 엄격한 분리는 **보안상 바람직**하며, 공용 사이트에는 오직 검증된 완료 콘텐츠만 흘러가도록 보증합니다.

- **검색엔진/크롤러 및 SEO:** standardai.co.kr에 제공되는 페이지들은 검색엔진에도 노출될 수 있습니다. 하루 한 키워드 뉴스라는 특성상, SEO를 고려해 meta 태그나 구조화된 데이터를 포함하는 것이 좋습니다. 이러한 부분은 사용자 UI 개발 가이드에서 강조하는 “현대적인 기능” 중 하나일 것입니다 ⁹⁶. 현재 생성되는 HTML이 간단한 포맷이라면, 추후 개선으로 meta description이나 SNS 미리보기(card) 등을 넣어야 할 수 있습니다. 이와 같은 작업은 Admin 생성 단계에서 미리 해두거나, 사용자 사이트에서 추가 스크립트를 통해 보강 가능합니다.

- **요약:** **admin.standardai.co.kr**는 **내부 편집 용도**로 Cloud Run 상에서 작동하며, Admin 팀만 접근 가능하고 토큰으로 보호됩니다. **standardai.co.kr**는 **외부 공개용 사이트**로 일반 독자들이 접근하며, Admin 시스템이 만들어낸 콘텐츠를 제공하는 역할입니다. 두 도메인은 물리적으로 분리되어 있어, 하나가 장애가 생겨도 다른 하나에 영향을 최소화합니다. (예를 들어 admin 사이트가 일시 다운되어도 기존에 게시된 user 사이트 콘텐츠는 정적이므로 그대로 볼 수 있습니다.) 최종 도메인 기준으로 볼 때, QualiJournal은 **이원화된 시스템**으로 운영되며, 편집과 발행이 admin 도메인에서, 열람이 standardai 도메인에서 이뤄집니다.

10. 개발자 및 운영자를 위한 전체 구조도 및 요약 체크리스트

마지막으로, 앞서 분석한 내용을 토대로 **퀄리저널 시스템의 전체 구조도**를 텍스트로 설명하고, 개발/운영 관점의 **체크리스트**를 정리합니다.

전체 아키텍처 구조 요약:

- **데이터 수집 단계:** 외부 뉴스/커뮤니티/블로그 소스 → (크롤러 & 수집 스크립트) → 원본 JSON들 → `rebuild_kw_from_today.py` → **작업본 JSON**(`selected_keyword_articles.json`) ⁴⁵
(수집 스크립트와 Admin 시스템의 연계는 아직 loose-coupling 상태: 수집 결과를 Admin이 읽는다.)
- **관리자 편집 단계:** **Cloud Run Admin 서비스**(`admin.standardai.co.kr`)
 - ↳ `index.html` (토큰 인증) → **FastAPI 백엔드**(`server_quali.py`)
 - ↳ `tools/` 스크립트 호출: `enrich_cards.py` (요약/번역) ¹¹, `make_daily_report.py` (보고서 생성) ²² 등
 - ↳ **데이터 업데이트:** 작업본 JSON 내 기사 필드 추가/수정, 승인 상태 수동 편집, 발행본 JSON 동기화 ³², config 임계값 수정 등
 - ↳ **결과물 저장:** `archive/` 폴더에 Markdown/HTML/JSON 출력 ²² + **GCS 업로드** (동일 파일들) ⁶¹

- **배포/공개 단계: 일반 사용자 웹** (standardai.co.kr)
 - ↳ GCS 아카이브로부터 최신 HTML 제공 (정적 호스팅) 또는 별도 CMS/페이지에 복사 게시
 - ↳ 독자들이 웹 페이지 열람 (모바일/PC 지원, 반응형 UI)
 - ↳ Cloud CDN/Cache 통해 콘텐츠 배포 최적화 가능 (선택)
- **CI/CD 및 인프라:** 개발자가 코드 변경 → GitHub에 push → **Actions CI** 빌드/테스트/배포 30
 - ↳ Cloud Run 새로운 Revision 생성, 트래픽 전환 125 82
 - ↳ 커스텀 도메인 (admin.standardai.co.kr)에서 새 버전 Admin UI 제공 (index.html 버전 업데이트로 확인) 120
 - ↳ GCS 버킷 (qualijournal-archive)에 일일 발행물 축적 (자동 백업)
 - ↳ Cloud Scheduler (미구현) or 수동으로 일일 작업 트리거/검증

(구조 다이어그램) - 텍스트 기반 표현:

```
[External Sources] -> {Collector Scripts} ->
[selected_keyword_articles.json]
      |                                   |
      v                                   v
[Admin UI (index.html) - token] -> (FastAPI Admin API) -> {tools scripts &
logic} -> [Archive files + GCS]
                                   |
                                   <---- JSON files (data/) -----> |
```

```
[Cloud Run Admin Service] --(custom domain)--> admin.standardai.co.kr
(secure, token)
[Cloud Storage Bucket] --(custom domain or manual publish)-->
standardai.co.kr (public, read-only)
```

(참고: 위 다이어그램은 Admin 시스템이 JSON을 받아 처리하고 결과물을 GCS에 업로드, 사용자 사이트는 GCS 또는 별도 경로에서 결과물을 제공하는 흐름을 묘사합니다.)

개발자/운영자 체크리스트: (시스템 유지보수와 일일 운영을 위한 주요 점검 사항)

- **코드 변경 및 배포 체크**
- [] Git 커밋/PR: 코드를 main 브랜치에 머지하기 전에 모든 변경사항 (특히 index.html UI 수정 등) 커밋이 반영되었는지 확인합니다 126 .
- [] CI 결과: GitHub Actions에서 **빌드와 테스트가 성공**했는지 확인합니다. 실패한 워크플로우가 있다면 배포되지 않으므로, 로그를 확인해 에러를 수정해야 합니다.
- [] Cloud Run 리비전: 배포 후 GCP에서 Cloud Run 서비스의 최신 Revision이 생성되었는지, **활성 트래픽 100%**가 할당되었는지 확인합니다 127 73 . 필요한 경우 CLI로 gcloud run revisions list 및 ... update-traffic 를 실행해 최신 버전으로 트래픽을 몰아줍니다 128 129 .
- [] 커밋 해시 검증: Cloud Run 서비스의 metadata.labels 내 **commit-sha** 라벨이 현재 최신 커밋 해시와 일치하는지 확인합니다 116 82 . 만약 다르다면 구버전이 배포된 것이니, 워크플로우 설정을 점검해야 합니다.
- [] Admin UI 버전 확인: 배포 완료 후 admin.standardai.co.kr/index.html 을 새로고침하거나 (또는 캐시를 우회하여) 열어봅니다. 페이지 소스에 표시된 버전 번호나 최신 수정사항(UI 텍스트 등)을 확인하여 **배포 반영 여부**를 검증합니다 120 118 .
- **일일 운영 체크 (발행 프로세스)**

- [] 데이터 수집 여부: 아침에 해당일 키워드에 대한 **데이터 수집이 완료**되었는지 확인합니다. 예를 들어 `/api/status`를 호출해 `selection_total` 값이 기대치(수십~수백 기사)인지 확인하거나, 수집 스크립트 로그를 점검합니다 88. 수집이 안 된 경우 Cloud Scheduler나 Cron이 제대로 작동했는지 살핍니다.
- [] 요약/번역 키 상태: 번역 API 키 (`API_KEY`)의 **유효성/쿼터**를 모니터링합니다. Admin UI에서 요약/번역 실행 시 에러 메시지가 뜨지 않는지 확인하고 48, 만약 "quota exceeded" 등의 메시지가 보이면 즉시 API 키 상태를 점검하거나 교체합니다.
- [] 승인 기사 수: 편집자가 충분한 기사를 승인했는지 (`approved=true` 개수) 확인합니다. `/api/status`의 `selection_approved`가 **최소 15 이상**인지, UI의 KPI에 게이트 통과(`gate_pass`)가 true로 표시되는지 봅니다 33. 만약 부족하면 `force_approve_top20.py` 등을 활용해 신속히 승인수를 맞추거나, 필요한 경우 게이트 임계값을 일시 조정합니다 (슬라이더 사용).
- [] 보고서 생성 결과: "일일 보고서 생성" 후 반환된 **Markdown 링크/파일**을 열어 내용이 올바른지 훑어봅니다 130 53. 모든 기사 제목/요약/출처가 포함됐는지, 형식이 깨지진 않았는지 확인합니다. 만약 일부 기사 데이터에 문제가 있어 보고서가 불완전하면, 백엔드가 생성한 파일을 수동 보완하거나 (아카이브 폴더에서 열어 수정) 다시 보고서 생성을 시도합니다 110 131.
- [] Export 기능 활용: 필요에 따라 **CSV Export**를 실행해 당일 선정 기사 리스트를 내려받고, Excel로 열어 한글 깨짐 등이 없는지 확인합니다 109 132. CSV는 내부 기록용으로 매일 보관하거나, 혹은 DB 마이그레이션 용도로 쌓아둘 수 있습니다.
- [] GCS 아카이브 업로드: 발행 후 Cloud Run 로그나 GCS 콘솔을 통해, **백업 버킷에 신규 파일들이 올라갔는지** 확인합니다 62 133. 예: `2025-10-26_탄소중립.md/.html/.json` 파일이 버킷에 존재하면 OK. 만약 없다면 Admin 컨테이너에 권한 문제가 있거나 업로드 코드에 버그일 수 있으므로, 빨리 원인 파악 후 수동으로 해당 파일을 가져와 저장합니다 21 98. (일일 백업 실패 시 당일치 자료 유실 가능성이 있으므로 중요한 체크 포인트입니다.)
- [] 사용자 사이트 게시: GCS에 파일이 있다면 `standardai.co.kr`에서 **실제 페이지 노출을** 확인합니다. 브라우저 캐시를 비우고 접속해서 최신 콘텐츠가 보이는지, 날짜/키워드가 맞는지, 이미지나 링크에 문제 없는지 검사합니다. 만약 사용자 사이트가 수동 게시 방식이라면, 다운로드한 HTML/MD 내용을 CMS에 올리거나 필요한 경로에 복사한 뒤, 페이지가 정상 출력되는지 확인합니다.

• 주기적 운영 점검 (주간/월간)

- [] 토큰/키 교체 주기: 보안을 위해 **Admin Token**이나 **API Key**를 주기적으로 변경할지 정책을 세우고, 교체 시에는 Cloud Run 환경변수와 GitHub Secrets를 모두 업데이트합니다 134. 교체 후 Admin 서비스 재배포를 잊지 않도록 합니다.
- [] 백업 용량 관리: GCS 아카이브 버킷에 데이터가 누적됨에 따라 용량과 비용을 모니터링합니다. 문서 권고대로 **Lifecycle 규칙**(예: 1년 지난 객체 삭제 또는 Coldline 전환)을 설정하여 비용을 최적화합니다 135 98. 또한 중요한 아카이브는 별도로 오프라인 백업(로컬 다운로드 등) 해둘지 검토합니다.
- [] 로그 모니터링 및 보안: Cloud Run 로그를 주기적으로 검토하여 **오류 징후나 비정상 접근 시도**가 없는지 확인합니다 83. 특히 인증 실패 로그가 반복되거나, Admin API에 토큰 없이 접근한 내역이 다수 발견되면 (공격 시도) IP 차단이나 토큰 변경 등의 조치가 필요합니다. 로그에 민감정보(토큰 값 등)가 남지 않도록 개발시에 주의했지만, 혹시 예외 케이스가 없는지도 살핍니다 136.
- [] 성능 및 비용 점검: 시스템 사용량이 늘면 Cloud Run 인스턴스가 scale out 할 수 있으므로, **동시성 설정과 메모리 사용량**을 모니터링합니다. 필요시 인스턴스당 동시 처리 요청 수를 낮추거나 (기본 80 → 1로) 수평 확장을 막아, 상태 공유 이슈를 회피할 수 있습니다. 반대로 성능 향상을 원하면 DB 도입이나 캐시 도입 등의 계획을 세웁니다.
- [] 향후 기능 로드맵 검토: 현 시스템이 안정적으로 돌아가는지를 전제로, **추가 요구사항**(예: AI 요약 고도화, 키워드 추천 알고리즘, 다중 사용자 관리 등)을 백로그로 관리합니다 137. 정기 미팅 때 구현 우선순위를 논의하고, 새로운 개발 착수 전에는 반드시 전체 구조에 미칠 영향(특히 JSON→DB 전환 등)을 평가합니다.

以上が QualiJournal 시스템 개발 파일에 대한 **최종 통합 설명서**입니다. 전체 구조와 흐름, 그리고 세부 구현 및 운영상의 유의점을 망라하여 정리하였으며, 이는 개발자와 운영자 모두에게 유용한 지침이 될 것입니다. 시스템이 `admin.standardai.co.kr`와 `standardai.co.kr`라는 두 축으로 분리되어 있지만, 결국 콘텐츠를 중심으로 하나로 이

어지는 만큼, 본 문서를 통해 **전반적인 맥락과 구성요소들의 연관성**을 잘 파악하시기 바랍니다. 운영 과정에서 새로운 이슈가 발견되거나 요구사항 변경이 있을 경우, 본 자료를 기반으로 원인을 진단하고 구조를 개선하는 데 활용할 수 있을 것입니다. 앞으로도 정기적인 점검과 꾸준한 개선을 통해 QualiJournal 시스템이 안정적으로 “하루 한 키워드” 뉴스를 제공할 수 있기를 기대합니다. 1 64

1 2 3 4 5 6 7 8 9 10 11 12 14 15 16 18 19 22 23 24 25 26 28 29 30 31 33 34 35
36 37 38 46 47 48 51 52 53 54 55 56 57 60 66 70 72 75 76 78 83 84 85 87 88 89 90 91 92
93 99 100 101 102 103 104 105 106 108 109 110 113 114 115 121 130 131 132 134 136 137 1011_3QualiJournal

관리자 시스템 최종 가이드북.pdf

file:///file-6Hu18vVrjQweSb5SZz6a6M

13 17 32 41 42 44 45 49 50 95 1004_2A고도화report.pdf

file:///file-Ko3WmavWUUXUmLMC1s6fso

20 21 27 39 40 58 59 61 62 63 65 67 68 69 71 74 77 86 98 111 112 122 123 124 133 135

1014_2QualiJournal 관리자 시스템 가이드.pdf

file:///file-V1gqDLDq7V1uefyJAegEJS

43 1003_5A퀄리뉴스를 전면적으로 리팩터링하여 안정적이고 확장 가능한 시스템으로 구축하려면.pdf

file:///file-C1X8pMSvAVd6Xs392FEnwq

64 94 96 97 1015_C_퀄리저널 사용자 UI 디자인 및 기능 고도화 개발자 가이드.pdf

file:///file-8Tx32iWj7TymAjvEmaWcjL

73 79 80 81 82 107 116 117 118 119 120 125 126 127 128 129 1022_1Cloud Run 배포 확인 및 최신 리비전 트래픽 적용 가이드.pdf

file:///file-7X7JDAg5PPzVgjeeR8DG8V